

Reviewer Pack — Hyperbolicity Lower Bound on Tseitin Resolution Length

Entry 844cf870afe5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 23:56:56 UTC

Hyperbolicity Lower Bound on Tseitin Resolution Length

Entry ID: 844cf870afe5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 23:56:56 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Resolution Length

Statement:

For a Tseitin formula on a graph G , its resolution length is $\geq 2^{\Omega(\delta(G))}$, where $\delta(G)$ is Gromov hyperbolicity. For graphs with $\delta(G) \leq 1$ (tree-like), resolution length is polynomial in n .

Rationale (proposer’s reasoning):

Hyperbolicity captures geometric constraints on shortest paths; graphs with higher $\delta(G)$ force resolution proofs to traverse exponentially more clauses due to non-tree-like detours, amplifying proof complexity.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd7d68fa0de0abe6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def negate(lit):
    return -lit if lit > 0 else -lit + 1

def dpll_with_memoization(clauses, assignment, memo=defaultdict(lambda: None)):
    key = tuple(sorted(assignment.items()))
    if key in memo:
        return memo[key]

    if not clauses:
        return True
    if any(not any(lit in assignment and assignment[lit] == val for lit, val in clause) for clause in clauses):
        return False

    literal = next(lit for lit in range(1, max(abs(lit) for clause in clauses) + 1) if lit not in assignment)

    def extend_assignment(lit):
        new_assignment = assignment.copy()
        new_assignment[lit] = True
        return new_assignment

    def extend_assignment_negate(lit):
        new_assignment = assignment.copy()
        new_assignment[lit] = False
        return new_assignment

    if dpll_with_memoization([clause for clause in clauses if literal not in clause and -literal not in clause]) or dpll_with_memoization([clause for clause in clauses if literal not in clause and -literal not in clause]):
        memo[key] = True
        return True

    memo[key] = False
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n, m):
        graph = {i: [] for i in range(1, n + 1)}
        edges = set()
        while len(edges) < m:
            u, v = random.sample(range(1, n + 1), 2)
```

```

        if u != v and (u, v) not in edges and (v, u) not in edges:
            graph[u].append(v)
            graph[v].append(u)
            edges.add((u, v))
    return graph

def gromov_hyperbolicity(graph):
    n = len(graph)
    if n < 4:
        return 0

    def triangle_distance(a, b, c):
        return min(len(graph[a] & graph[b]), len(graph[b] & graph[c]), len(graph[c] & graph[a]))

    max_delta = -math.inf
    for a in range(1, n + 1):
        for b in range(a + 1, n + 1):
            for c in range(b + 1, n + 1):
                delta = (triangle_distance(a, b, c) + triangle_distance(a, c, b) - triangle_distance(b, a, c))
                max_delta = max(max_delta, delta)
    return max_delta

def tseitin_formula(graph):
    n = len(graph)
    literals = {i: (2 * i - 1, 2 * i) for i in range(1, n + 1)}
    clauses = []

    for u in range(1, n + 1):
        if not graph[u]:
            clauses.append([literals[u][0]])
        else:
            clauses.append([-literals[u][0]] + [literals[v][1] for v in graph[u]])

    for u in range(1, n + 1):
        for v in range(u + 1, n + 1):
            if v not in graph[u]:
                clauses.append([literals[u][1], literals[v][0]])
                clauses.append([literals[u][0], literals[v][1]])

    return clauses

def resolution_length(clauses):
    return len(clauses) if dpll_with_memoization(clauses, {}) else float('inf')

n = random.randint(5, 40)
m = random.randint(n - 1, n * (n - 1) // 2)
graph = generate_random_graph(n, m)
delta_G = gromov_hyperbolicity(graph)
clauses = tseitin_formula(graph)

resolution_length_value = resolution_length(clauses)
conjecture_holds = True
counterexample = ""

if delta_G >= 1 and resolution_length_value < 2 ** delta_G:
    conjecture_holds = False

```

```

        counterexample = f"Graph with n={n}, m={m} has  $\delta(G)={\delta\_G}$  but resolution length {resol
    elif delta_G <= 1 and not (5 * (n - 1) ** 3 // 6 <= resolution_length_value <= 4 * n ** 3):
        conjecture_holds = False
        counterexample = f"Graph with n={n}, m={m} has  $\delta(G)={\delta\_G}$  but resolution length {resol

    return {
        "metric_name": "Resolution Length",
        "metric_value": resolution_length_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06a868ed.py", line 133, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06a868ed.py", line 105, in run_trial
    delta_G = gromov_hyperbolicity(graph)
            ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06a868ed.py", line 76, in gromov_hyperb
    delta = (triangle_distance(a, b, c) + triangle_distance(a, c, b) - triangle_distance(b, c, a)) / 2
            ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06a868ed.py", line 70, in triangle_dist
    return min(len(graph[a] & graph[b]), len(graph[b] & graph[c]), len(graph[c] & graph[a]))

```

~~~~~^~~~~~  
TypeError: unsupported operand type(s) for &: 'list' and 'list'

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to TypeError in hyperbolicity calculation, preventing data collection |  
next: Fix the list intersection operator error in gromov\_hyperbolicity implementation

## 11. Audit log (LLM calls)

**Total LLM calls:** 11

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 110458       |
| 2  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 54705        |
| 3  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 111380       |
| 4  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 54011        |
| 5  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 45962        |
| 6  | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 25908        |
| 7  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 26509        |
| 8  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 15048        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 48106        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16785        |
| 11 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 14811        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 523683 ms total latency. Provider mix: {'ollama\_remote': 11}

(full prompt+response transcripts available in `research/audit/844cf870afe5.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/844cf870afe5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/844cf870afe5.tar.gz` (if generated)